

TRINNITY VISERON SYSTEM

MANUAL DE OPERAÇÃO HUMANA

v5.0 — Todos os passos que uma pessoa precisa executar em todo o sistema

Ø=ÜQ Pedro Costa — Comandante Supremo
Ø=Üx Trinnity Hurtado — Rainha Arquiteta

Gerado automaticamente em 03/08/2026, 22:52:49

1. RESULTADO DO BARRIDO AUTOMÁTICO (varredura dos squads AIOX)

Uma varredura completa do sistema foi orquestrada pelos squads AIOX e supervisionada por Pedro Costa e Trinnity Hurtado. Foram encontradas e corrigidas as seguintes falhas críticas:

Área	Falha encontrada	Correção aplicada
Dashboard WebOS	webos.js com erro de sintaxe (interface morta)	Sintaxe corrigida — dashboard volta a funcionar
Forge Git	Rotas Express 5 inválidas (crash no arranque) + git push vazio	Rotas corrigidas + git-receive-pack/upload-pack reais
Configuração	.env nunca era carregado (IA cloud toda em modo simulado)	dotenv/config carregado nos servidores
AgentSpawner	Caminho de minds.json quebrado em modo dev	Resolução robusta (cwd + dirname)
IA	4.756 mentes usavam provider 'openai' (violava regra Ollama)	Provider padrão agora é ollama (local)
Scripts npm	call:start, jarvis:start, asno:start, omniroute:start quebrados	startServer exportado + guarda contra null
WebAppGenerator	Token sites gerados com sintaxe inválida + portas 3000 colidindo	Interpolação corrigida + portas 4100+
HyperLearning	Inteligência explodia x6 por ciclo (valor sem sentido)	Crescimento +5% com teto de 1.000.000
tvstools	Comando 'transcrever' nunca funcionava (precedência)	Parênteses corrigidos
Produção	Porta 3000 fixa ignorava PORT (crash em Railway/Render)	Agora respeita process.env.PORT
Segurança	.env ia para Docker/pkg/backups; senha n8n fixa	.dockerignore criado; .env removido de pkg/backups; senha via env
Mentes	Missão de 5.000+ mentes não era cumprida (4.756)	Regeneradas: 5.000 mentes (342 históricas + 4.658 sintéticas)

Estado final verificado: build OK, lint OK, testes 10/10, arranque com 5.370 agentes totais (5.000 mentes + 246 arquétipos + 114 batallón + ~10 núcleo).

2. PREPARAÇÃO INICIAL (executar UMA VEZ)

2.1 — Instalar ferramentas

- Instalar Node.js 20 LTS (<https://nodejs.org>) — necessário para compilar e rodar o sistema.
- Instalar Git (<https://git-scm.com>) e configurar: ``git config --global user.name/user.email``.
- Clonar/copiar o repositório para uma pasta sem espaços, ex.: `C:\Trinnity-Viseron-System`.

2.2 — Instalar dependências

```
npm install
```

```
cd mobile && npm install && cd ..
```

- Se não houver `node_modules` no projeto, este comando cria tudo que é preciso.

2.3 — Configurar variáveis de ambiente (.env)

- Copiar o modelo: ``Copy-Item .env.example .env`` e editar com o Bloco de Notas.
- IA LOCAL (obrigatório p/ funcionar sem custo): instalar Ollama e baixar modelos:

```
ollama pull llama3:8b
ollama pull qwen3:9b
ollama pull deepseek-coder
```

- IA NUVEM (opcional, para raciocínio complexo): preencher `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`, `XAI_API_KEY` no `.env`.
- Serviços opcionais: `TWILIO_ACCOUNT_SID/AUTH_TOKEN` (chamadas), `RENDER_API_KEY` (deploy), `HOSTALIA_FTP_*` (FTP), `CLOUDFLARE_*` (CDN/DNS).

2.4 — Compilar o sistema

```
npm run build
```

- Gera a pasta dist/ com o sistema pronto para produção.

3. OPERAÇÃO DIÁRIA (todos os dias)

3.1 — Ligar o sistema

```
npm start
```

- OU modo desenvolvimento (recarrega sozinho ao editar código):

```
npm run dev
```

3.2 — Verificar que está tudo OK

- Abrir o WebOS (interface do sistema): <http://localhost:3000/dashboard.html>
- Abrir o relatório PDF em tempo real: <http://localhost:3001/report/pdf>
- Ver o relatório completo (agentes, tokenomics, linhagens): <http://localhost:3001/report/comprehensive-pdf>
- Conferir o log: ``Get-Content data/system.log -Tail 100``.
- Confirmar que o total de agentes é ~5.370 e que a evolução avança (+5% por ciclo).

3.3 — Comandos de voz / chat

- Usar o painel de voz no dashboard para falar com JARVIS/ASNO (requer microfone e, p/ chamadas reais, credenciais Twilio + URL pública).

4. OPERAÇÃO SEMANAL (1x por semana)

4.1 — Backup

```
npm run backup
```

- Gera um .zip em backups/ com configuração, dados, memória e código.
- O .env NÃO entra no zip por segurança — guarde as chaves em um cofre separado.
- Para automatizar todo dia às 03:00, executar uma vez:

```
npm run backup:schedule
```

4.2 — Verificações de qualidade

```
npm run lint
```

```
npm test
```

- O lint deve terminar sem erros e os testes devem mostrar 10/10 PASS.

4.3 — Revisar memória de longo prazo

- A memória cresce em database/memory/ltn.json (pode passar de 50 MB). Revisar tamanho e limpar backups antigos em database/memory/backups/ se necessário.

5. MANUTENÇÃO MENSAL (1x por mês)

- Atualizar dependências com cuidado: ``npm audit`` e atualizar apenas o necessário.
- Limpar backups antigos (o script já apaga com mais de 30 dias) e logs grandes (data/system.log).
- Regenerar as mentes se quiser variar os agentes sintéticos: ``npx tsx scripts/generateMinds.ts``.
- Revisar a pasta generated-apps/ (apps gerados por IA) e remover o que não for usar.

- Verificar os relatórios em data/reports/ (cycle_N.json, evolution_log.json) e arquivar PDFs importantes.
- Testar o build completo de ponta a ponta: `npm run init:full` (build + backup + start).

6. PUBLICAÇÃO / DEPLOY (quando for publicar)

6.1 — Publicar código no GitHub

```
npm run deploy:github
```

- Confirma que o remote está certo: `git remote -v` (o correto é github.com/ViseronSystem/trinnity-viseron-system).
- Nunca commitar o .env — ele já está no .gitignore.

6.2 — Publicar o site na Vercel (site estático)

```
npm run deploy:vercel
```

- Se for a primeira vez: `npx vercel login` e autorizar no navegador.

6.3 — Publicar a API/backend na Render

```
npm run deploy:render
```

- Antes, configurar em .env: RENDER_API_KEY e RENDER_SERVICE_ID.
- Na Render, adicionar as variáveis de ambiente (PORT, e as chaves de IA) no painel do serviço.

6.4 — Publicar por FTP (Hostalia)

```
npm run deploy:hostalia
```

- Antes, configurar em .env: HOSTALIA_FTP_HOST, HOSTALIA_FTP_USER, HOSTALIA_FTP_PASS, HOSTALIA_FTP_PATH.
- No servidor de produção, rodar `npm start` com PORT atribuída pelo provedor (o sistema respeita process.env.PORT).

7. BACKUP E RECUPERAÇÃO

7.1 — Fazer backup

```
npm run backup
```

- Backups ficam em backups/YYYY-MM-DD_HHMMSS.zip (retidos 30 dias).

7.2 — Recuperar de um backup

- Descompactar o zip em uma pasta limpa.
- Restaurar manualmente o .env (não está no zip por segurança).
- Rodar `npm install` e `npm run build`, depois `npm start`.

7.3 — Agendar backup automático

```
npm run backup:schedule
```

- Cria a tarefa 'TVS-DailyBackup' no Agendador do Windows (03:00).

8. APP MÓVEL (Android APK / iOS IPA)

8.1 — Preparar o ambiente

- Android: instalar Android Studio + SDK 34 e criar um emulador (ou conectar um aparelho com depuração USB).
- iOS: só funciona em macOS com Xcode instalado.

8.2 — Desenvolvimento com Expo

```
npm run mobile:start
```

8.3 — Gerar o APK Android

```
npm run build:android
```

• Para gerar APK final para a Play Store, é preciso conta EAS: ``cd mobile && npx eas login`` e ``npx eas build --platform android``.

8.4 — Gerar o IPA iOS (macOS)

```
npm run build:ios
```

8.5 — Configurar o servidor no app

- O app conecta ao servidor padrão definido em `mobile/app.json` !' `extra.tvsServerUrl` (ex.: `http://192.168.1.10:3000` para a rede local).
- Dentro do app, a tela de Configurações permite trocar o IP do servidor sem recompilar.

9. IA LOCAL E NUVEM

9.1 — Local (padrão, sem custo)

- Todas as 5.000 mentes usam o modelo local via Ollama (provider 'ollama').
- Verificar se o Ollama está rodando: ``ollama list`` e ``ollama serve``.
- Escolher outro modelo local: editar `config/tvs.config.json` !' `models.local` ou preencher `OLLAMA_HOST` no `.env`.

9.2 — Nuvem (raciocínio complexo)

• Preencher as chaves em `.env` e reiniciar o sistema. O `ModelRouter` usa nuvem quando a tarefa é complexa e local quando é privada/rápida.

9.3 — OmniRoute (gateway com 290+ provedores, opcional)

```
npm run omniroute:start
```

- Requer o pacote global: ``npm install -g omniroute`` (ou rode ``npm run omniroute:install``).

10. SERVIÇOS EXTERNOS OPCIONAIS

10.1 — Chamadas telefônicas (Twilio)

- Preencher `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN` e `TWILIO_PHONE_NUMBER` no `.env`.
- Para o streaming de voz funcionar, expor o sistema com URL pública (ex.: ngrok) e definir `PUBLIC_HOSTNAME` no `.env`.
- Instalar a dependência: ``npm install twilio``.

10.2 — Automação n8n

- O sistema tenta iniciar n8n na porta 5678. Instalar n8n: ``npm install -g n8n`` ou via `docker-compose`.

10.3 — Banco vetorial Qdrant (memória semântica)

• Sem Qdrant, a memória usa fallback em RAM (funciona, mas não persiste embeddings). Para ativar: ``docker run -d -p 6333:6333 qdrant/qdrant``.

10.4 — Home Assistant (casa inteligente)

• Preencher `HOME_ASSISTANT_URL` e `HOME_ASSISTANT_TOKEN` no `.env`. Dispositivos são controlados por entidades do tipo `'domain.entity_id'` (ex.: `light.sala`).

10.5 — Sistema de chamadas / JARVIS / ASNO avulsos

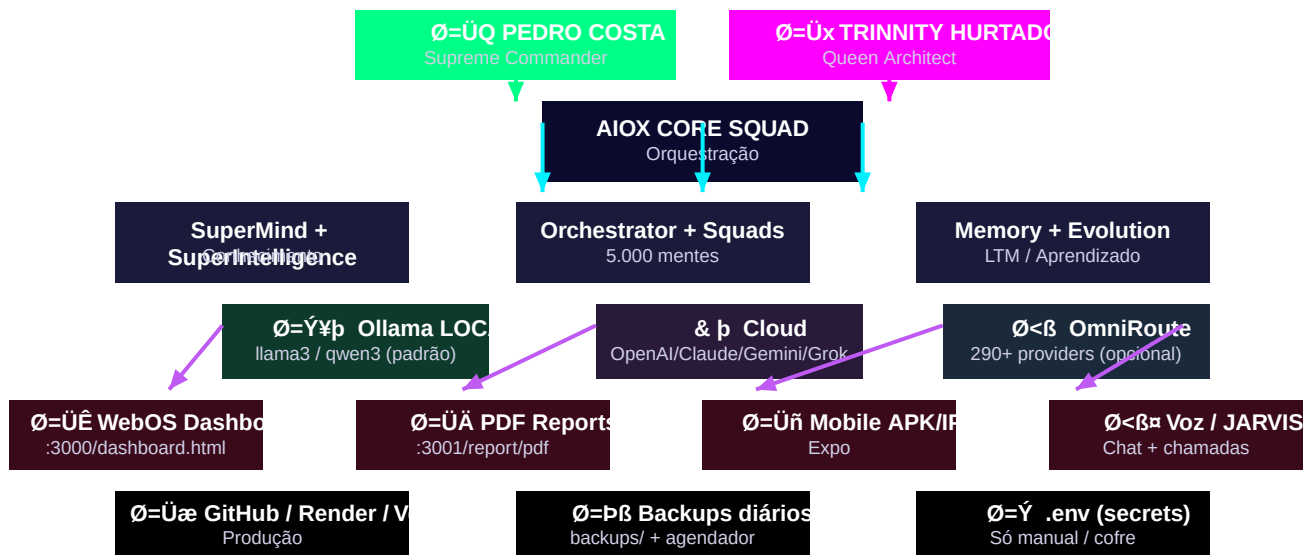
```
npm run call:start
```

```
npm run jarvis:start
```

```
npm run asno:start
```

11. DIAGRAMA DE OPERAÇÃO

Fluxo de comando e operação do Trinnity Viseron System v5.0:



12. PLATAFORMA CODE — OPERAR E CRIAR VISERON

A CODE Platform transforma o sistema numa consola de operação real: criar mentes VISERON, executar agentes com IA local e monitorizar o AIOX. Tudo a partir do navegador.

12.1 — Aceder à plataforma CODE

- Abrir o WebOS: <http://localhost:3000/dashboard.html> e clicar no ícone #(& p) CODE.
- A app tem 4 separadores: Console (comandos), Criar VISERON (blueprints), Agentes e Apps LLM.

12.2 — Comandos do console

```
help # lista de comandos
```

```
status # estado do sistema em tempo real
```

```
agents # lista todas as mentes registadas
```

```
blueprints # 7 blueprints prontos (BizAnalyst, DataMind, etc.)
```

```
create NovaMente Rol # cria uma mente VISERON nova
```

```
run <id> < tarefa> # executa um agente com IA local (Ollama)
```

12.3 — Apps LLM (catálogo awesome-llm-apps)

- O separador Apps LLM oferece 8 aplicações prontas inspiradas no repositório awesome-llm-apps: Deep Research, Local RAG, Mixture of Agents, Multi-Agent Team, Self-Evolving, Always-On Briefing, Voice RAG e Generative UI.
- Cada app cria uma mente especializada que pode ser executada de imediato com um clique.
- As skills ficam em skills/vendor/awesome-llm-apps/ e podem ser consultadas em `GET /api/skills`.

12.4 — Autoria e donos do sistema

- Pedro Costa — Comandante Supremo & Criador (clearance tvs_creator, autoridade absoluta).
- Trinnity Hurtado — Rainha & Arquiteta Chefe (clearance tvs_architect, soberania técnica).
- Todos os squads executivos e de arquitetura são liderados por eles; a autoria fica registrada em data/Viseron_Autoria_e_Propriedade.md.

12.5 — Monitorização AIOX

- GET /api/code/aiox mostra o nível de conhecimento AIOX, o estado do cérebro de Pedro/Trinnity e as métricas de memória.
- O AutoLearningEngine corre um ciclo de aprendizagem a cada 30 min e atualiza automaticamente o conhecimento do sistema.
- Para auditoria completa com squads AIOX-1..5: npm run audit:arkom.

13. TABELA DE COMANDOS RÁPIDOS

O que fazer	Comando
Iniciar o sistema	npm start
Modo desenvolvimento (hot reload)	npm run dev
Compilar	npm run build
Testes	npm test
Verificação TypeScript	npm run lint
Backup manual	npm run backup
Agendar backup diário 03:00	npm run backup:schedule
Publicar (tudo)	npm run deploy
Publicar GitHub	npm run deploy:github
Publicar Vercel	npm run deploy:vercel
Publicar Render	npm run deploy:render
Publicar Hostalia (FTP)	npm run deploy:hostalia
APK Android	npm run build:android
IPA iOS (macOS)	npm run build:ios
App Expo em desenvolvimento	npm run mobile:start
Inicialização completa	npm run init:full
Sistema de chamadas (Twilio)	npm run call:start
OpenJarvis (IA local Stanford)	npm run jarvis:start
ASNO (WhatsApp + casa inteligente)	npm run asno:start
OmniRoute (gateway 290+ provedores)	npm run omniroute:start
Gerar as 5.000 mentes	npx tsx scripts/generateMinds.ts
Ver relatório PDF	http://localhost:3001/report/pdf
Ver WebOS	http://localhost:3000/dashboard.html
Plataforma CODE (criar/operar VISERON)	http://localhost:3000/dashboard.html !' #(\p CODE
Estado AIOX (monitorização)	GET /api/code/aiox
Instalar skills (catálogos)	npm run skills:install
Auditoria ARKOM/AIOX	npm run audit:arkom